effective_rank

October 6, 2026

1 Effective Rank (Roy & Vetterli, 2007)

Implementación del *effective rank*, referenciado en el capítulo de ViT (bloque A.1: similitud + rango efectivo entre cabezas de atención).

Dada una matriz A con valores singulares $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_Q \geq 0$ ($Q = \min(M, N)$):

$$p_k = \frac{\sigma_k}{\|\sigma\|_1}, \qquad H(p_1, \ldots, p_Q) = -\sum_k p_k \log p_k, \qquad \operatorname{erank}(A) = \exp\{H(p_1, \ldots, p_Q)\}$$

con la convención $0 \log 0 = 0$. Cumple $1 \leq \operatorname{erank}(A) \leq \operatorname{rank}(A) \leq Q$.

[3]:
```python
import math

import torch


def effective_rank(A: torch.Tensor) -> torch.Tensor:
    """
    Effective rank de Roy & Vetterli (2007): erank(A) = exp(H(p)),
    donde p_k = sigma_k / ||sigma||_1 son los valores singulares de A
    normalizados y H es su entropía de Shannon (con la convención 0*log(0) = 0).

    Soporta batches: A puede tener shape (..., M, N), y el rango efectivo
    se calcula sobre las últimas dos dimensiones, devolviendo shape (...,).
    """
    singular_values = torch.linalg.svdvals(A)  # (..., Q)
    p = singular_values / singular_values.sum(dim=-1, keepdim=True)

    log_p = torch.where(p > 0, torch.log(p), torch.zeros_like(p))
    entropy = -(p * log_p).sum(dim=-1)

    return torch.exp(entropy)
```

1.1 Pruebas con dos matrices de referencia

1. **Identidad** 4×4: los 4 valores singulares son iguales (=1), la distribución p_k es uniforme y $H(p) = \log 4$, por lo que $\operatorname{erank}(I_4) = 4$.

2. **Matriz circulante Hermitiana** del ejemplo del propio paper (Sección 2.3), con valores singulares $(1+|\rho|)^2, 1-|\rho|^2, 1-|\rho|^2, (1-|\rho|)^2$, cuya forma cerrada es $\text{erank}(A) = \exp\{2H(\frac{1+|\rho|}{2}, \frac{1-|\rho|}{2})\}$.

[4]:
```python
def binary_entropy(p: float) -> float:
    q = 1 - p
    terms = [x * math.log(x) for x in (p, q) if x > 0]
    return -sum(terms)
```

[5]:
```python
n = 4
A = torch.eye(n)
erank = effective_rank(A).item()
expected = float(n)
print(f"[identidad {n}x{n}]     erank = {erank:.6f}  (esperado {expected:.6f})")
assert math.isclose(erank, expected, rel_tol=1e-5)
```

```
[identidad 4x4]     erank = 4.000000  (esperado 4.000000)
```

[6]:
```python
rho = 0.5
A = torch.tensor([
    [1, rho, rho**2, rho],
    [rho, 1, rho, rho**2],
    [rho**2, rho, 1, rho],
    [rho, rho**2, rho, 1],
])

erank = effective_rank(A).item()
expected = math.exp(2 * binary_entropy((1 + abs(rho)) / 2))
print(f"[circulante rho={rho}] erank = {erank:.6f}  (esperado {expected:.6f})")
assert math.isclose(erank, expected, rel_tol=1e-5)
```

```
[circulante rho=0.5] erank = 3.079201  (esperado 3.079201)
```

1.2 Rango efectivo de las cabezas de atención de un modelo entrenado

Modelo ViT entrenado en MNIST (`model.pt`, solo pesos). El checkpoint no trae `config.json`, así que la arquitectura se infiere de las shapes de los tensores guardados: `hidden_size=48`, `num_hidden_layers=4`, `num_attention_heads=4` (head_size=12), `intermediate_size=192`, `patch_size=4`, `image_size=32`, `num_channels=3`, `num_classes=10`, sin `faster_attention` (cabezas separadas con `.query`/`.key`/`.value`).

[7]:
```python
import sys
from pathlib import Path

project_root = Path.cwd().resolve().parents[1]
print(f"Project root: {project_root}")
sys.path.append(str(project_root))

import torch
```

```
from modules.model import ViTForClassfication
```

```
Project root: /Users/valefeve/Projects/interpretability of neural
networks/interpretable-transformers
```

[8]:
```
config = {
    "hidden_size": 48,
    "num_hidden_layers": 4,
    "num_attention_heads": 4,
    "intermediate_size": 192,
    "hidden_dropout_prob": 0.0,
    "attention_probs_dropout_prob": 0.0,
    "initializer_range": 0.02,
    "image_size": 32,
    "patch_size": 4,
    "num_channels": 3,
    "num_classes": 10,
    "qkv_bias": True,
    "use_faster_attention": False,
}

model = ViTForClassfication(config)
state_dict = torch.load("model.pt", map_location="cpu")
model.load_state_dict(state_dict)
model.eval()
```

[8]:
```
ViTForClassfication(
  (embedding): Embeddings(
    (patch_embeddings): PatchEmbeddings(
      (projection): Conv2d(3, 48, kernel_size=(4, 4), stride=(4, 4))
    )
    (dropout): Dropout(p=0.0, inplace=False)
  )
  (encoder): Encoder(
    (blocks): ModuleList(
      (0-3): 4 x Block(
        (attention): MultiHeadAttention(
          (heads): ModuleList(
            (0-3): 4 x AttentionHead(
              (query): Linear(in_features=48, out_features=12, bias=True)
              (key): Linear(in_features=48, out_features=12, bias=True)
              (value): Linear(in_features=48, out_features=12, bias=True)
              (dropout): Dropout(p=0.0, inplace=False)
            )
          )
          (output_projection): Linear(in_features=48, out_features=48,
bias=True)
```

```
          (output_dropout): Dropout(p=0.0, inplace=False)
        )
        (layernorm_1): LayerNorm((48,), eps=1e-05, elementwise_affine=True,
bias=True)
        (mlp): MLP(
          (dense_1): Linear(in_features=48, out_features=192, bias=True)
          (activation): NewGELUActivation()
          (dense_2): Linear(in_features=192, out_features=48, bias=True)
          (dropout): Dropout(p=0.0, inplace=False)
        )
        (layernorm_2): LayerNorm((48,), eps=1e-05, elementwise_affine=True,
bias=True)
      )
    )
  )
  (classifier): Linear(in_features=48, out_features=10, bias=True)
)
```

[9]:
```
last_block = model.encoder.blocks[-1]
heads = last_block.attention.heads

query_weights = torch.stack([head.query.weight.detach() for head in heads])  #␣
 ↪(num_heads, head_size, hidden_size)
key_weights = torch.stack([head.key.weight.detach() for head in heads])
value_weights = torch.stack([head.value.weight.detach() for head in heads])

print(f"{len(heads)} cabezas de atención en el último bloque, cada matriz con␣
 ↪shape {tuple(query_weights.shape[1:])}")
print("\nQuery weights por cabeza:")
print(query_weights)
print("\nKey weights por cabeza:")
print(key_weights)
print("\nValue weights por cabeza:")
print(value_weights)
```

```
4 cabezas de atención en el último bloque, cada matriz con shape (12, 48)

Query weights por cabeza:
tensor([[[-0.2291, -0.1780,  0.3583,  …,  0.2037, -0.2656,  0.1339],
         [-0.1131,  0.0604, -0.2410,  …,  0.0264, -0.5287,  0.0772],
         [ 0.5420, -0.3695, -0.3649,  …, -0.1642,  0.4750,  0.4939],
         …,
         [-0.3528,  0.2980,  0.2032,  …,  0.1384, -0.2366,  0.0783],
         [ 0.1911, -0.3010,  0.0577,  …,  0.2775,  0.0901,  0.0967],
         [ 0.1895,  0.0067, -0.1698,  …,  0.1204, -0.0930, -0.1797]],

        [[-0.8964,  0.1974, -0.0554,  …, -0.5615,  0.6035, -0.4337],
         [-0.6277,  0.0586, -0.0784,  …, -0.0959,  0.3321, -0.5254],
```

```
          [ 0.1581,  0.2315,  0.1012,  …, -0.0458, -0.1851,  0.1605],
          …,
          [-0.0437,  0.3215, -0.3140,  …, -0.1167, -0.5517,  0.4733],
          [-0.0539,  0.2547, -0.0547,  …,  0.1062, -0.3981,  0.3137],
          [-0.5351, -0.1965,  0.1460,  …,  0.9475,  0.7021, -0.0982]],

         [[ 0.0741,  0.1169,  0.1290,  …,  0.2435, -0.3743,  0.2731],
          [ 0.5981,  0.3445,  0.1994,  …,  0.4719,  0.0278,  0.5444],
          [-0.2580,  0.4481,  0.1712,  …,  0.0336, -0.0863,  0.5223],
          …,
          [ 1.0807, -0.6552, -0.0378,  …,  0.3778, -0.5078,  0.0944],
          [-0.1125,  0.2049, -0.0100,  …, -0.1053, -0.7020,  0.0490],
          [ 1.2270, -0.2896, -0.0948,  …,  0.0722, -0.6623, -0.7507]],

         [[ 0.9692,  0.3667, -0.6436,  …,  0.0895, -0.2472,  0.4660],
          [ 1.1710, -0.3265,  0.1204,  …,  0.4170,  0.1823, -0.7197],
          [-1.3513,  0.5064, -0.2300,  …, -0.3741, -0.2480,  0.8911],
          …,
          [-0.4280, -0.2390, -0.1720,  …,  0.5655,  0.2998, -0.5494],
          [-0.0305, -0.2002,  0.1406,  …, -0.6878,  0.1835, -0.8483],
          [-0.1923, -0.3438,  0.7482,  …, -0.1553,  0.2983, -0.7898]]])

Key weights por cabeza:
tensor([[[ 0.5170,  0.5528, -1.2192,  …,  0.4825, -0.2300, -0.2132],
          [ 0.0974,  0.1989, -0.7482,  …,  0.3961,  0.0770, -0.2793],
          [-0.2188, -0.4492,  1.3598,  …, -0.3355, -0.1759, -0.6585],
          …,
          [-0.3703,  0.1310, -0.2187,  …, -0.0212,  0.6115,  0.4746],
          [ 0.7753,  0.5975, -0.6377,  …,  0.3563, -0.2952,  0.0385],
          [ 0.6531,  0.4163, -0.6021,  …,  0.3783, -0.4418, -0.0613]],

         [[ 0.0620, -0.1736, -0.7132,  …, -0.4628,  0.1668,  0.7306],
          [-0.7988,  0.3806, -0.4694,  …,  0.0873, -0.0530, -0.1909],
          [ 1.1397,  0.1452,  0.0972,  …,  0.2178, -0.5737,  0.0649],
          …,
          [-0.6559,  0.7229, -0.3088,  …,  0.0521,  0.5341,  0.3774],
          [ 0.8824,  0.1921, -0.2468,  …,  0.2641, -0.0255,  0.4459],
          [-0.7759,  0.2109, -0.5049,  …,  0.7531,  0.7545, -0.1571]],

         [[ 0.4896,  0.5651,  0.1451,  …,  0.6834,  0.5366, -0.0032],
          [ 0.5031, -0.1172, -0.2494,  …, -0.1436,  0.2170,  0.0444],
          [ 0.1128, -0.1332,  0.0866,  …, -0.2009,  0.0990,  0.0573],
          …,
          [ 0.7631, -0.5565, -0.8241,  …,  0.3036, -0.8116,  0.0567],
          [ 0.5005,  0.3014, -0.6765,  …,  0.1690, -0.3409, -0.3080],
          [ 0.4859, -0.6129, -0.8335,  …, -0.0810, -1.0120, -0.1346]],

         [[ 0.1730, -0.8809,  0.8435,  …,  0.2586,  0.3817, -0.4069],
```

```
          [-0.6410,  0.9121,  0.1001,  …,  0.1881,  0.2528, -0.1918],
          [ 0.5674, -1.1610, -0.0196,  …, -0.2086, -0.5112,  0.0709],
          …,
          [-0.4442,  1.2862, -0.4011,  …, -1.4887,  0.8944,  0.1841],
          [-0.4532,  0.2718,  0.3098,  …, -0.2569,  1.0802, -0.1314],
          [-0.2778,  1.0746, -0.6129,  …, -0.0778, -0.0855,  0.3507]]])

Value weights por cabeza:
tensor([[[ 3.1111e-01,  5.2884e-02, -2.1066e-01,  …,  1.1565e-01,
           -7.8639e-01, -3.0933e-01],
          [-6.8402e-01,  3.7760e-01, -3.8059e-01,  …,  1.9874e-01,
            4.4074e-02,  1.4936e-01],
          [ 3.6845e-01, -1.4799e-01, -1.0896e-01,  …, -1.9313e-01,
           -4.6251e-02,  6.2984e-01],
          …,
          [-8.2233e-02, -1.1228e-01, -8.5007e-02,  …, -1.7216e-01,
           -2.6463e-01, -3.7797e-04],
          [ 4.3443e-02,  7.7285e-02,  5.7799e-02,  …, -2.3011e-01,
            5.5053e-02,  1.2826e-03],
          [-1.7958e-01, -7.8438e-02, -2.6720e-01,  …,  4.7022e-02,
           -9.7859e-02, -1.7982e-01]],

         [[-7.0571e-02, -2.8057e-01,  5.8609e-01,  …, -3.0434e-01,
            7.2040e-03, -3.6635e-01],
          [-6.7114e-03, -1.6085e-01, -2.7551e-01,  …, -1.8316e-01,
            1.4548e-02,  1.5917e-01],
          [-9.6320e-01,  5.0211e-01, -4.3528e-01,  …,  3.6797e-01,
            1.0318e-02, -6.8898e-01],
          …,
          [ 1.0394e-01, -1.5175e-02,  8.8916e-01,  …,  1.2391e-01,
            6.2832e-01,  2.1109e-01],
          [ 1.3493e-01,  7.5314e-02,  2.5615e-01,  …, -4.7225e-02,
           -7.6649e-02, -8.1711e-02],
          [ 3.1093e-01, -3.2708e-01,  5.1824e-01,  …, -7.5758e-02,
            2.1805e-01,  7.3382e-01]],

         [[ 1.7066e-02,  1.9307e-01, -1.1793e-01,  …, -4.4911e-02,
            1.3118e-01, -2.7076e-01],
          [-5.6523e-01, -5.1762e-01,  1.0066e+00,  …, -3.1624e-01,
            2.4221e-01, -7.2943e-01],
          [-5.5467e-01, -7.9991e-01, -2.2693e-01,  …,  3.9178e-02,
            4.2057e-01, -7.1883e-02],
          …,
          [-5.0069e-01, -9.4723e-01,  9.0579e-02,  …, -1.6265e-01,
            7.2331e-01,  9.7275e-01],
          [-2.2860e-02,  1.6078e-01,  3.2909e-02,  …,  1.8555e-01,
            5.1904e-03, -3.1317e-01],
          [ 1.2217e-01, -1.6215e-01,  1.3604e-01,  …, -4.1882e-02,
```

```
          8.0518e-02,  3.6905e-01]],

        [[ 2.3839e-01, -1.0902e-01, -1.1054e-01,  …,  1.9815e-01,
         -2.1546e-01, -6.1345e-01],
         [ 2.9932e-02, -1.3977e-01, -1.6419e-02,  …,  2.3897e-01,
          2.5927e-03, -2.1747e-02],
         [ 2.6646e-01,  1.3516e-01, -1.2842e-01,  …, -1.1542e-01,
          1.8436e-02, -1.5014e-01],
         …,
         [-3.5286e-01,  5.4195e-01,  8.3713e-01,  …, -5.6094e-02,
          6.3574e-01,  5.9976e-01],
         [-3.7401e-01, -2.0863e-01, -5.9346e-01,  …, -1.0060e-01,
          9.3345e-02,  8.8709e-01],
         [-2.1488e-01, -9.0659e-03, -1.3656e-01,  …, -5.5425e-02,
         -5.7126e-02, -3.0925e-01]]])
```

[10]:
```
erank_query = effective_rank(query_weights)
erank_key = effective_rank(key_weights)
erank_value = effective_rank(value_weights)

for h in range(len(heads)):
    print(f"head {h}: erank(Q)={erank_query[h]:.4f}  erank(K)={erank_key[h]:.
 ↪4f}  erank(V)={erank_value[h]:.4f}")
```

```
head 0: erank(Q)=8.8387  erank(K)=7.5238  erank(V)=10.5189
head 1: erank(Q)=8.5091  erank(K)=7.7325  erank(V)=10.3649
head 2: erank(Q)=9.4800  erank(K)=8.6640  erank(V)=10.3762
head 3: erank(Q)=7.4286  erank(K)=6.2817  erank(V)=10.6534
```

[11]:
```
model.encoder.blocks
```

[11]:
```
ModuleList(
  (0-3): 4 x Block(
    (attention): MultiHeadAttention(
      (heads): ModuleList(
        (0-3): 4 x AttentionHead(
          (query): Linear(in_features=48, out_features=12, bias=True)
          (key): Linear(in_features=48, out_features=12, bias=True)
          (value): Linear(in_features=48, out_features=12, bias=True)
          (dropout): Dropout(p=0.0, inplace=False)
        )
      )
      (output_projection): Linear(in_features=48, out_features=48, bias=True)
      (output_dropout): Dropout(p=0.0, inplace=False)
    )
    (layernorm_1): LayerNorm((48,), eps=1e-05, elementwise_affine=True,
bias=True)
    (mlp): MLP(
```

```
      (dense_1): Linear(in_features=48, out_features=192, bias=True)
      (activation): NewGELUActivation()
      (dense_2): Linear(in_features=192, out_features=48, bias=True)
      (dropout): Dropout(p=0.0, inplace=False)
    )
    (layernorm_2): LayerNorm((48,), eps=1e-05, elementwise_affine=True,
bias=True)
  )
)
```

[ ]: